RESEARCH AND IMPLEMENTATION REVIEW

Further Lean Proof Simplifications

New deltas after the existing DiophantineConsistency roadmap

Prepared for Vladimir Reshetnikov

26 October 2026

Source baseline

`VladimirReshetnikov/ProveIt`

`553a5f7a463cbaab8800ff8148fef0bc5a9e1b22`

Central finding

Several remaining proofs can be simplified by recovering the invariant that decides a branch before analyzing that branch. The strongest concrete example removes support-dependent reasoning for the first 21 of 22 quadratic-position rows. Other opportunities replace a helical induction by period closure, derive a period bound directly from strip geometry, recover a pairing branch from its output, and consume already available semantic equations in the internal-natural layer.

Scope of the novelty claim

The recommendations were not found as proposals in the audited corpus, including all 21 research packets present at the closing snapshot and the twelve earlier incoming archives. All earlier proposals are treated as already available, including deferred and uncompiled suggestions. This is a repository-relative novelty claim, not a claim that the underlying mathematical facts are new.

A compiled report, Lean examples, reviewable patches, provenance, an exclusion ledger, and replay instructions accompany this document. Validation levels are stated individually; no whole-project build or cumulative saving against a hypothetical fully refactored tree is claimed.

Contents

1 Findings and recommended order **2**
1.1 What the measurements do and do not mean . . . 2

2 Source baseline and exclusion audit **3**
2.1 Important families deliberately excluded . . . 3
2.2 Nearest-prior comparison for the retained proposals . . . 4

3 P1: prove the span before inspecting most supports **4**
3.1 The mathematical cutoff . . . 4
3.2 Concrete replacement . . . 5
3.3 Why this survives the counterfactual baseline . . . 5

4 P2: use the helix as a period in two dimensions **6**
4.1 Replace an induction by a group law . . . 6
4.2 Maintenance benefit and limits . . . 7

5 P3: derive the period lower bound from strip width **7**
5.1 A stronger theorem with geometric premises . . . 7
5.2 A simpler client and a dependency improvement . . . 8

6 P4: recover the pairing branch from the output **8**
6.1 A threshold inside each square shell . . . 8
6.2 Only two branches remain . . . 9
6.3 Charge the new lemma honestly . . . 9

7 P5: use the constructor equation in the list-head bound **9**

8 P6: remove two local geometric-sum inductions **10**
8.1 Use the theorem whose statement already matches . . . 10
8.2 Derive the upper bound without its own induction . . . 10

9 P7: build the internal order from its strict part **11**
9.1 The redundant order-law reconstruction . . . 11
9.2 Definitional compatibility is the real cost . . . 11
9.3 When to adopt it . . . 12

10 Validation, measurements, and replay **12**
10.1 Raw source-size observations . . . 13
10.2 Evidence levels . . . 13
10.3 Replay in the pinned repository . . . 14
10.4 Recommended implementation sequence . . . 14

11 Artifact guide and limitations **14**

A Primary source map **15**

B Supplement: direct polarization in the Sidon argument **16**

1 Findings and recommended order

The best first change is the proof of `NativeQuadraticPositions.index_span`. Its large case analysis can be reduced for a mathematical reason: the target position of every row except the last is already below the required bound, before anything is known about its support. The second small, robust change is `row_zero_of_helix`: an integer multiple of a translation period moves any point to row zero, so mathlib's period laws replace the hand-written positive and negative induction.

The review retains seven directions, with different adoption costs. Several are small local improvements. The order-construction proposal is an interface change and belongs last, because it changes the definitional form of non-strict order even though its meaning is proved equivalent.

ID	Target	New contribution after the prior roadmap
P1	Quadratic-position span	Discard support reasoning for rows 0–20; analyze only the five supported pairs in row 21.
P2	Helical row-zero theorem	Regard the helix as one period in $\mathbb{Z}^2$ and use closure under integer multiples.
P3	Strip period bound	Prove $L + R \leq N$ geometrically; obtain $N > 2$ without reconstructing a halted run.
P4	Max-square pairing	Recover the pairing branch from the common output and shell maximum before proving injectivity.
P5	Internal list-head bound	Use `head_cons` and monotonicity directly, eliminating the conversion to and from additive slack.
P6	Polynomial range preliminaries	Reuse `mul_geom_sum`; derive the geometric upper bound algebraically without another induction.
P7	Internal-natural order	Construct order from a strict total order, with explicit adapters for the old non-strict relation.

P1, P2, P4, P5 and P6 are candidates for proof-local changes. P3 is a stronger geometric lemma whose main benefit is a simpler dependency structure. P7 requires a deliberate compatibility decision. Precise validation outcomes and the difference between an exact-statement check and an entire production-module check appear in Section 10.

1.1 What the measurements do and do not mean

The pinned production directory contains 445 Lean modules and 75,502 physical source lines, counting the files under `Lean/DiophantineConsistency`. These figures describe the inspected tree, including check modules. They are not a count of proof lines or of irreducible mathematical work.

The user asks us to assume the earlier proposals are already applied. There is no supplied integrated tree implementing every such proposal. Consequently, raw before/after line counts below compare against the actual pinned source only. They must not be added together and advertised as an incremental saving over that hypothetical tree. The more reliable incremental claims are structural: fewer support-dependent obligations, fewer logically possible branches, no custom integer induction, no unnecessary slack witness, and lower-level premises for a period bound.

2 Source baseline and exclusion audit

The repository was read at the fixed commit `553a5f7a463c`. The toolchain file specifies Lean 4.32.0. The root Lake manifest pins mathlib to `81a5d257c8e410db227a6665ed08f64fea08e997`. Library facts in this report refer to that exact version.

The initial direct exclusion corpus consisted of the six placed research packets `2026-10-26a` through `2026-10-26f`, plus twelve ZIP files in `docs/incoming`. The audit records all 323 files in those placed packets and all 2,405 ZIP members. Of the ZIP members, 2,067 have relevant text extensions. These are inventory counts, not claims that each member was read line by line. Main report bodies, relevant notes and prototypes, optional suggestions, existing exclusion ledgers, and targeted full-text searches were used to test the retained ideas. Archive bytes were checked against the Git blob identifiers for the pinned snapshot. Old proposal scripts were not executed.

A closing comparison used commit `e652d21c413de59e4821f6e8e9b6a3a077bf5bfe`. At that snapshot the research directory has packets a–u and incoming contains only its README. The six a–f directory tree hashes are unchanged. Packets g–r contain the twelve previously audited archives: 625 retained nonadministrative files are byte-identical and six CSVs differ only by CRLF-to-LF normalization. The three genuinely added packets s–u were then reviewed, including optional suggestions, relevant prototypes, and affected source patches. Their nearest-prior distinctions are recorded in `CLOSING_AUDIT.md`. No retained recommendation was removed by that comparison. The production modules targeted by this report are unchanged between the two source snapshots.

The incoming README was used for routing and scope. The audit does not assert that every historical manuscript transitively linked by that large index was re-read. The accompanying `corpus_manifest.json`, `report-index.json`, and `exclusion_ledger.json` make the direct comparison reproducible.

2.1 Important families deliberately excluded

The initial ledger has 83 grouped exclusions. Its closing supplement adds eleven specific entries, including variants and reinforcements of earlier families; these are not eleven disjoint new families. In particular, this report does not re-propose:

- list lookup APIs, assignment naturality, shared scanners, recursor congruence, numeric Boolean reflection, and single-source rule schedules;
- shared residual squares, generic semiring gate proofs, topological DAG invariants, power-label certificates, register cuts, the common free-carrier tail, or nine-coordinate witness packaging;
- coefficient envelopes, finite-fiber convolution, support envelopes, rotation as point-mass convolution, canonical cyclic carries, or packing as an algebra homomorphism;
- Pell congruence up to sign, the existing odd-index criterion, exact integer half-binomial recovery, natural perturbation bounds, positive-pair base monotonicity, or rational-root preservation;
- fused decoder specifications, internal constructor views, unique-existence induction, recursive-graph preservation/fusion, model-to-derivation transport, or monotone local certificate calculus.
- the closing packets' compositional satisfaction normalization, exact library list-code

identification, local coefficient fibres, centered interval induction, first-failure prefix contracts, stronger native-rank envelope, or shared-edge Wang row construction.

Some of these were promising initial candidates and were discarded only after opening the incoming archives. Several exclusions occur only in ancillary notes. For example, packet f already suggests a finite membership interface for `NativeQuadraticPositions.lean`; the newer index-span proposal must therefore do more than replace a chain of `Finset.mem_insert` cases by a prettier dispatcher.

2.2 Nearest-prior comparison for the retained proposals

ID	Closest earlier idea	Remaining new step
P1	Packet f's finite support membership interface.	Even a perfect membership interface is unnecessary in 21 rows: the target bound alone proves the result for arbitrary pair weights.
P2	Existing periodic-plane source; u's centered induction for interval propagation.	Apply a global integer-period group law to the two-dimensional helix; no interval induction is required.
P3	Earlier geometric transport; t/u's first-failure prefix helper.	Compare a positive period with the entire seam-free strip width after the walls are constructed, then derive the lower bound before halting interpretation.
P4	Canonical pairing witnesses and inverse-coordinate graph projection.	Recover which pairing branch occurred from an output threshold after equal maxima have been established.
P5	Earlier internal arithmetic/order interfaces; s's formula-specification normalization.	Consume the existing constructor equation at this inequality client; remove a redundant inequality-to-witness-to-inequality detour.
P6	General coefficient and arithmetic cleanup suggestions.	Exact reuse of `mul_geom_sum` and an algebraic proof of the particular geometric bound in `PolynomialRangeBounds.lean`.
P7	Definable-induction abstractions; s's two specification edits in this file.	Construct the order instance from strict membership; account for the changed reduction behavior of $\leq$. Earlier specification edits leave the instance intact.

3 P1: prove the span before inspecting most supports

3.1 The mathematical cutoff

The source target is `index_span` in `NativeQuadraticPositions.lean`. Write

$$T(k) = 349(k+1), \qquad E(k,i) = T(k) - W(i),$$

where subtraction is natural-number subtraction and $W(i)$ is the pair weight. The desired upper bound is $E(k,i) \le 7516$ for supported pairs in each of 22 rows.

For $k < 21$, the support hypothesis is irrelevant:

$$E(k,i) \le T(k) \le 349 \cdot 21 = 7329 < 7516.$$

This is valid for every natural $W(i)$, including $W(i) > T(k)$, because truncated subtraction still satisfies $T(k) - W(i) \le T(k)$. There is no need to establish positivity of the exponent, to unfold the pair enumeration, or to inspect a support entry.

Only $k = 21$ remains. Its support is the literal five-element set

$$\{59, 18, 64, 21, 0\}.$$

The corresponding pair weights are $271, 204, 267, 200, 162$. Since $T(21) = 7678$, the five exponents are as follows.

Pair index	59	18	64	21	0
Weight	271	204	267	200	162
Exponent	7407	7474	7411	7478	7516

The upper bound remains sharp. In particular, this refactor does not replace the precise degree argument by a looser asymptotic estimate. The supported final pair with index zero still attains 7516.

3.2 Concrete replacement

With the original theorem signature, the proposed body is:

```
intro k i hi
by_cases hk : k.val < 21
· have ht : target k ≤ 7516 := by dsimp [target]; omega
  exact (Nat.sub_le _ _).trans ht
· have hk21 : k = 21 := Fin.ext (by have := k.isLt; omega)
  subst k
  change i ∈ {59, 18, 64, 21, 0} at hi
  simp only [Finset.mem_insert, Finset.mem_singleton] at hi
  rcases hi with rfl | rfl | rfl | rfl | rfl <;> decide
```

The fixture supplied with the archive extracts the actual source definitions needed by this statement. Its extraction manifest records their hashes. It does not import the old `index_span` theorem as an assumption.

3.3 Why this survives the counterfactual baseline

The raw private theorem declaration has 92 physical lines, including its signature; the candidate declaration has 11 under the same counting convention. The raw whole file

changes from 258 to 177 lines. Those differences are useful for reviewing the patch but are not the asserted post-roadmap saving.

The durable gain is removing 131 of the 136 support-dependent bound obligations. After a previous finite-membership refactor, those 131 cases might already have very short proofs, but they would still depend on literal support data. With this cutoff, modifications to those first 21 supports do not require new evidence for this particular span bound. Only the final row is sensitive to its entries.

The five-entry premise must remain in the final row. For example, the unsupported pair with index 7 would give exponent 7597. Extending the final-row claim to every pair would be false. The file's `kernel_top_coefficient` and `kernel_natDegree` statements retain their original numerical meaning.

4 P2: use the helix as a period in two dimensions

4.1 Replace an induction by a group law

The theorem `row_zero_of_helix` in `WangTiles942HelicalQuotient.lean` starts from

$$T(x + w, t - 1) = T(x, t)$$

and proves

$$T(x, t) = T(x + wt, 0)$$

for all integers x, t, w and arbitrary output type. The source proves this by `Int.induction_on`, separately treating positive and negative time.

Let $F(x, t) = T(x, t)$. Its hypothesis says precisely that $(w, -1)$ is a period of $F : \mathbb{Z}^2 \to \alpha$. Integer multiples of a period are periods. Taking the multiple t gives

$$(x, t) + t(w, -1) = (x + tw, 0).$$

The needed closure theorem is already present as `Function.Periodic.zsmul`.

```
theorem row_zero_of_helix {α : Type*} (T : ℤ → ℤ → α) (w : ℤ)
    (hhelix : ∀ x t, T (x+w) (t-1) = T x t) (x t : ℤ) :
    T x t = T (x+w*t) 0 := by
  have hp : Function.Periodic (fun p : ℤ × ℤ => T p.1 p.2)
      (w, -1) := by
    rintro ⟨a,b⟩
    simpa only [Prod.fst_add, Prod.snd_add, sub_eq_add_neg]
      using hhelix a b
  simpa [zsmul_eq_mul, mul_comm] using (hp.zsmul t (x,t)).symm
```

The compiled artifact proves the exact original statement in an isolated namespace. It requires neither positivity of w nor a finite modulus or vertical period. It also does not assume that the helix is a least period.

4.2 Maintenance benefit and limits

This is a direct application of an existing library law, with no new framework to maintain. The same viewpoint makes combinations of geometric periods explicit: if $(w, -1)$ and $(0, H)$ are periods, then $H(w, -1) + (0, H) = (wH, 0)$ is a period. The current short corollaries need not all be rewritten; the recommendation is to replace the duplicated integer induction where the library law already exactly matches the statement.

The report does not claim new periodic geometry. `WangTiles942Periodic.lean` already represents several period facts. The new delta is recognizing the two-coordinate hypothesis as the input to `Periodic.zsmul` at this specific proof boundary.

5 P3: derive the period lower bound from strip width

5.1 A stronger theorem with geometric premises

An `InitialStrip T N L R` has a left seam at $-L$, a right seam at R, and no seam in the open interval $(-L, R)$. Its fields include $0 < L$, $L \leq N$, and $2 \leq R$. The exact tile descriptions and `wall_north` are in `WangTiles942InitialStrip.lean`; the uniform non-seam interior fact is `initial_vertical_nonseam` in `WangTiles942StripGeometry.lean`.

Proposition 5.1. *If `InitialStrip T N L R` holds and T has positive period N, then*

$$L + R \leq N.$$

Proof. Translate the left seam at $-L$ by one period. The point $-L + N$ is another seam. Since $N > 0$, it lies strictly to the right of $-L$. If $N < L + R$, it also lies strictly to the left of R, contradicting the seam-free interior. Thus $N \geq L + R$. □

In the project application, positivity of N need not be a new parameter: it follows from $0 < L \leq N$. Equality $N = L + R$ is allowed. At equality the translated left seam reaches the right endpoint, which is deliberately outside the open interior.

The new statement can be added near the existing strip geometry:

```
theorem initial_width_le_period {T : ℤ → Tile} {N L R : ℕ}
    (h : InitialStrip T N L R)
    (hperiod : Function.Periodic T (N : ℤ)) : L + R ≤ N := by
  have hN : 0 < N := lt_of_lt_of_le h.left_pos h.left_le
  by_contra hwidth
  apply (initial_vertical_nonseam h
    (-(L : ℤ) + (N : ℤ)) (by omega) (by omega)).1
  exact (congrArg Tile.north (hperiod (-(L : ℤ)))).trans
    h.wall_north.1
```

The supplied fixture and integration sketch state all variables and use the actual source structure. The central proof only transports the north color along one periodicity equality and applies `initial_vertical_nonseam`.

5.2 A simpler client and a dependency improvement

Since $L \geq 1$ and $R \geq 2$, the width inequality yields $N \geq 3$. Therefore a marked word's period exceeds two directly from its initial strip and horizontal periodicity.

The existing `marked_period_gt_two` in `WangTiles942SoundnessChecks.lean` instead obtains a halting time smaller than the period and rules out halting at the first two actual run times. The new proof makes this particular lower bound independent of that run reconstruction and calibration. It does not claim to remove all machine-data imports from the tile language, and the check file may still need its other imports for unrelated checks.

The current client is already short. The point is to prove a stronger fact at the correct earlier layer, not to manufacture a large line-count claim. This lemma is also reusable when a future application needs room for the entire initial strip, rather than merely $N > 2$.

The supplied two-file patch changes the old client body from nine lines to one, but adds two reusable declarations. Its total source size grows by eleven lines. A generic seam-gap kernel and its numerical consequence have been checked. The larger exact-definition fixture timed out without diagnostics under resource contention, and its retry was interrupted; the production adapter and its equality-boundary example remain uncompiled candidates. This distinction is preserved in the validation records.

6 P4: recover the pairing branch from the output

6.1 A threshold inside each square shell

`ObjectPairingInjective.lean` proves injectivity for the max-square pairing under the original assumptions `CommSemiring`, `LinearOrder`, `CanonicallyOrderedAdd`, and `IsStrictOrderedRing`. In particular, every element is nonnegative. These hypotheses are suitable for internal naturals:

$$\mathrm{pair}(a,b) = \begin{cases} b^2 + a, & a < b, \\ a^2 + a + b, & a \geq b. \end{cases}$$

The existing shell bounds imply that equal outputs have equal maxima, provided the stated discrete successor-order hypothesis is retained. The source then considers all four combinations of $a < b$ and $c < d$.

Once the maximum is known, the output itself determines the branch.

Lemma 6.1. *Let* $m = \max(a,b)$. *Then*

$$\mathrm{pair}(a,b) < m^2 + m \quad \Longleftrightarrow \quad a < b.$$

Proof. If $a < b$, then $m = b$ and $b^2 + a < b^2 + b$. Otherwise $m = a$, and $a^2 + a + b \geq a^2 + a$ because $b \geq 0$. These are exactly the two sides of the equivalence. □

The threshold lemma itself does not need the discrete successor-order hypothesis. That hypothesis is still needed in the existing argument identifying the square shell from equal outputs. It must not be removed from the injectivity theorem.

6.2 Only two branches remain

From $\text{pair}(a, b) = \text{pair}(c, d)$, the existing proof first obtains

$$\max(a, b) = \max(c, d).$$

The threshold equivalence then gives $a < b \leftrightarrow c < d$. One case split on $a < b$ suffices:

- In the true branch, equal maxima give $b = d$; additive cancellation gives $a = c$.
- In the false branch, equal maxima give $a = c$; additive cancellation gives $b = d$.

The key Lean step is:

```
have hbranch : a < b ↔ c < d := by
  rw [← pair_lt_shell_cut a b, ← pair_lt_shell_cut c d, he, hm]
```

This removes both mixed-branch contradictions. It is different from the earlier canonical-certificate proposal: that proposal computes witnesses from input order; this one reconstructs branch information from the common output and shell.

6.3 Charge the new lemma honestly

The consumer's source becomes shorter, but the threshold lemma also occupies maintained lines. With the helper charged, the prepared full-file patch changes 104 physical lines to 100. Thus this is principally a simplification of the proof's logical structure. Reporting only the reduction in `pair_eq_pair_iff` would overstate the net saving.

The checked extraction retains the original semiring/order class assumptions and all helper facts needed by the theorem. It is not a proof merely specialized to ordinary natural numbers. Applying the production patch still warrants rebuilding the internal-natural client, since that is where the abstract class hypotheses are instantiated.

7 P5: use the constructor equation in the list-head bound

The source `ObjectListHead.listHead_le` proves

$$\text{listHead}(\mathit{code}) \leq \mathit{code}$$

over the internal-natural carrier of an arbitrary model of the supplied ZF axioms. It already splits the code into zero or a cons. In the nonzero case it converts `left_le_pair` into an additive slack witness, constructs the same slack on the desired side, proves the code nonzero, unfolds `listHead`, and reassociates the addition.

The existing equations in `ObjectNumericWire.lean` make that detour unnecessary:

$$\text{head}(\text{cons}(a, b)) = a + 1, \qquad \text{cons}(a, b) = \text{pair}(a, b) + 1.$$

Together with `left_le_pair`, the entire nonzero argument is monotonicity of addition:

$$a \leq \text{pair}(a, b) \implies a + 1 \leq \text{pair}(a, b) + 1.$$

The prepared replacement rewrites the nonzero code using `zero_or_cons`, invokes `head_cons`, uses the definition of `cons`, and applies `add_le_add`. No new helper is required. The zero case is unchanged. The full-file patch changes 129 physical lines to 121.

```
  · rw [hc, ObjectNumericWire.head_cons]
    exact add_le_add (left_le_pair H _ _) (le_refl 1)
```

This recommendation assumes the earlier internal ordered-semiring interfaces are already available, exactly as requested. It does not re-propose their construction. It also preserves the nonstandard-model scope: the proof uses equations and an order law valid in the internal carrier, not an identification of that carrier with external $\mathbb{N}$.

The generic fixture checks the argument from the exact constructor-equation interface. The archive separately labels the production patch and its replay requirement. A generic fixture should not be mistaken for a rebuild of all of `ObjectListHead.lean` and its object-theory dependencies.

8 P6: remove two local geometric-sum inductions

8.1 Use the theorem whose statement already matches

The first lemma in `PolynomialRangeBounds.lean` is

$$(B-1)\sum_{i=0}^{d-1} B^i = B^d - 1.$$

Its proof is a short induction, but its statement is already exactly `mul_geom_sum` specialized to integers. The replacement is:

```
theorem geometric_identity (B : ℤ) (d : ℕ) :
    (B - 1) * (∑ i ∈ range d, B ^ i) = B ^ d - 1 :=
  mul_geom_sum B d
```

This identity requires no positivity hypothesis on B; the source interface remains as general as before. An explicit import of `Mathlib.Algebra.Ring.GeomSum` documents the dependency.

8.2 Derive the upper bound without its own induction

Set $S = \sum_{i<d} B^i$. When $B \geq 2$, we have $S \geq 0$ and

$$(B-2)S \geq 0, \qquad (B-1)S = B^d - 1.$$

Subtracting these expressions gives $S \leq B^d - 1$. Hence

$$\sum_{i=0}^{d} B^i = S + B^d < 2B^d.$$

The proof is uniform in d, including $d = 0$. The Lean replacement expands the last summand, supplies the nonnegative product, and closes the resulting polynomial arithmetic:

```
theorem geometric_lt_two_top {B : ℤ} (hB : 2 ≤ B) (d : ℕ) :
    (∑ i ∈ range (d + 1), B ^ i) < 2 * B ^ d := by
  rw [sum_range_succ]
  have hs : 0 ≤ ∑ i ∈ range d, B ^ i :=
    sum_nonneg (fun i _ => pow_nonneg (by omega : 0 ≤ B) i)
  nlinarith [mul_geom_sum B d,
    mul_nonneg (show 0 ≤ B - 2 by omega) hs]
```

This is a modest cleanup with a clear proof boundary, not a new coefficient-envelope framework. The later `abs_eval_lt` and `eval_gt_top_power` clients keep the same hypotheses and conclusions. The large symbolic power remains symbolic; no numerical power expansion is needed.

9 P7: build the internal order from its strict part

9.1 The redundant order-law reconstruction

`ObjectNaturalOrder.lean` already has available the substantive facts about membership between internal naturals: irreflexivity, transitivity, and trichotomy, imported from the earlier development. It then manually constructs a `LinearOrder` on `Natural H`, repeatedly relating membership and equality.

The pinned mathlib provides `linearOrderOfSTO` for a strict total order. Registering the existing strict-order facts and invoking that constructor can replace much of the repeated order-law verification. The proposal keeps the underlying subtype, membership relation, arithmetic operations, and nonstandard-model interpretation.

9.2 Definitional compatibility is the real cost

The old non-strict relation unfolds to membership or underlying equality. The standard constructor uses equality of subtype elements or the strict relation. These propositions are equivalent, but their expressions do not reduce to the same term:

$$(\mathrm{mem}(x.val, y.val) \vee x.val = y.val) \quad \Longleftrightarrow \quad (x = y \vee \mathrm{mem}(x.val, y.val)).$$

Commuting a disjunction and converting subtype equality are proof steps, not definitional reduction.

An initial experiment tried to preserve both order relations by `rfl`. Restoring all the order fields consumed the intended simplification, so that variant is not recommended. The retained candidate uses the standard constructor directly and adds an explicit bridge

$$x \leq y \quad \Longleftrightarrow \quad \mathrm{LeRel}(H, x.val, y.val).$$

The prepared candidate has five small boundary conversions in the same source file. The prepared full-file candidate decreases from 175 to 160 physical lines, including the bridge and conversions.

9.3 When to adopt it

This is the highest-risk proposal in the report because downstream proofs may rely on unfolding $\leq$ by reduction. Keep it separate from the proof-local patches. First check all direct clients of `ObjectNaturalOrder`, looking especially for `change`, `rfl`, and explicit unfolding of the order instance. Prefer a small stable bridge lemma at those boundaries.

The generic prototype verifies the order construction and compatibility statement for an abstract carrier with the corresponding strict-order facts. That does not establish compatibility of every existing object-language module. Adopt P7 only after the downstream rebuild shows that the total adapter cost is smaller than the removed boilerplate. It remains useful to present the idea, but it should not block adoption of P1–P6.

10 Validation, measurements, and replay

Lean checks used version 4.32.0 (compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`) and the pinned mathlib revision where applicable. Every row below has a successful final standalone check and a matching source hash. The seven headline production candidates have not been compiled as complete production modules. The supplementary Sidon candidate is the full-module exception.

ID	Successful check	Remaining integration scope
P1	Exact-data fixture	Full `NativeQuadraticPositions` and its numerical-degree consumers.
P2	Exact theorem statement	Complete helical-quotient module and existing quotient clients.
P3	Generic integer core	Actual tile adapter and larger exact-definition fixture. The latter timed out; its retry was interrupted.
P4	Exact generic source segment	Internal-natural instantiation and the complete object-language module.
P5	Generic equation interface	Complete object-theory dependency chain for `ObjectListHead`.
P6	Both exact statements	Complete polynomial-range module and unchanged range clients.
P7	Generic order interface	Actual order instance and all reduction-sensitive downstream clients.
Aux.	Sidon theorem and full module	No claim about complete project integration or other modules follows.

No successful check reports `sorryAx`. The printed axiom sets contain only the ordinary `propext`, `Quot.sound`, and, where used, `Classical.choice`. The full Sidon module compiled without diagnostics; its isolated theorem check supplies the corresponding axiom inspection. All eight delivered patches passed a joint `git apply --check` at the source baseline. No patch was applied to the repository.

10.1 Raw source-size observations

The following counts include new imports and helper lemmas in the whole-file comparisons. They are review aids against the actual pinned source, not additive savings after the hypothetical prior roadmap.

ID	Physical source lines	More useful structural observation
P1	258 to 177 in the file	Private declaration: 92 to 11. Only five of 136 supported pairs remain relevant to this bound.
P2	147 to 137 in the file	Replaced declaration block: 19 to 7; no custom integer induction.
P3	11 additional lines overall	Old client body: 9 to 1; stronger width theorem and earlier geometric consequence added.
P4	104 to 100 in the file	Two actual branch cases replace four combinations; the threshold helper is charged.
P5	129 to 121 in the file	The slack-witness and nonzero detour disappear.
P6	150 to 144 in the file	Two local inductions replaced by a matching theorem and algebra.
P7	175 to 160 in the file	Includes bridge and five local conversions; external conversion cost remains unknown.
Aux.	168 to 161 in the file	Sidon declaration: 26 to 17, with two explicit imports charged to the file.

10.2 Evidence levels

An **exact-statement extraction** copies a source theorem's mathematical signature into an isolated namespace and checks its replacement using the pinned library version. It establishes that theorem at the stated generality, but it does not check every imported module, namespace interaction, or dependent client in the repository.

An **exact-data fixture** also copies the needed literal definitions, with their hashes recorded. This is particularly useful for P1, where a finite table is essential only in the exceptional row. It must not import the theorem being replaced or assume the desired bound.

A **generic interface fixture** proves the reusable argument from the exact equations or relation laws exposed at a module boundary. This is the appropriate description for an object-theory adapter when the complete formalized-ZF dependency chain has not been rebuilt.

A **production candidate** is a reviewable full source file or patch. A patch that applies cleanly is not thereby Lean-checked. The machine-readable verification records distinguish application checks, source/extraction hashes, theorem checks, and full-module checks.

The ordinary axioms reported by `\#print axioms` are retained in the logs. The validation rejects proof holes such as `sorryAx`; it does not confuse classical logic already used by the development with an unproved mathematical lemma. No computational benchmark is claimed: the shared execution environment had substantial resource contention, so observed wall times are not evidence of performance improvements.

10.3 Replay in the pinned repository

The archive's `scripts/replay.py` checks the supplied standalone Lean files under a user-provided checkout via `lake env lean`. It records individual exit codes and logs. It does not silently install an alternate toolchain or change the repository. The report can be rebuilt with XeLaTeX using `scripts/build_report.sh`.

The source patch set is intentionally separable. Review a patch, apply it in an isolated worktree at the pinned commit, and run the relevant module check and its existing check module. For P7, extend the check to direct and then transitive order clients because the definitional interface changes. This targeted rebuild is a concrete remaining integration obligation, not a reason to claim the proposal has already been integrated.

10.4 Recommended implementation sequence

1. Apply P1 and P2 separately. Confirm that the exported theorem signatures, literal position data, and numerical degree facts are unchanged.
2. Apply P5 and P6. These use existing semantic equations and a matching library theorem, with no new mathematical assumptions.
3. Add P4's threshold lemma and replace the injectivity proof. Count the helper and consumer together when reviewing maintenance cost.
4. Add P3 at the strip-geometry layer and use it for the period lower-bound client. Keep claims about dependency reduction local to that theorem unless imports are actually reorganized and rebuilt.
5. Trial P7 independently. Retain it only if downstream compatibility conversions remain small.

11 Artifact guide and limitations

The ZIP is the complete deliverable. It contains the TeX source and compiled PDF, Lean proof examples, candidate patches, the exclusion audit, source and extraction manifests, recorded validation results, and scripts for replay and PDF generation. The README identifies the highest-confidence artifacts and the optional interface proposal.

The bundle does not claim that all earlier packets were physically applied in this checkout. It also does not establish a globally minimal proof development, a complete build of every module, or improved run time. The principal conclusion is narrower and testable: after excluding the existing roadmap, several specific proof obligations admit additional simplification, and the accompanying evidence states exactly how far each has been checked.

A Primary source map

All production links below refer to the pinned commit, not a moving branch.

Source	Relevant declaration or role
NativeQuadraticPositions.lean	`target`, `index_span`; kernel degree and top-coefficient consumers.
NativeQuadraticParent.lean	Definition of `rowSupportIndices`; native quadratic row data.
WangTiles942HelicalQuotient.lean	`row_zero_of_helix`; signed row and quotient construction.
WangTiles942Periodic.lean	Actual signed address and plane periodicity.
WangTiles942InitialStrip.lean	`InitialStrip`, wall colors and marked-strip extraction.
WangTiles942StripGeometry.lean	`initial_vertical_nonseam`; natural home for the width lemma.
WangTiles942SoundnessChecks.lean	Existing `marked_period_gt_two` client.
ObjectPairingInjective.lean	Abstract max-square pairing bounds and injectivity.
ObjectNumericWire.lean	`zero_or_cons`, `head_cons`, and literal cons semantics.
ObjectListHead.lean	`left_le_pair`, `listHead_le`, guarded list-head certificate.
PolynomialRangeBounds.lean	Geometric-sum lemmas and their polynomial range clients.
ObjectNaturalOrder.lean	Membership facts and the current `LinearOrder` instance.
SidonPositions.lean	Supplementary first-two-moments argument inside pair-weight injectivity.
Mathlib periodicity API	`Function.Periodic.zsmul`, period addition and subtraction.
Mathlib geometric sums	`mul_geom_sum`.
Mathlib strict-order classes	`linearOrderOfSTO`.

The exact source hashes and declaration locations are recorded in the evidence directory. The exclusion ledger gives separate archive-member paths for prior ideas. Keeping those two kinds of evidence separate prevents an old proposal, a source definition, and a newly checked replacement from being mistaken for one another.

B Supplement: direct polarization in the Sidon argument

The archive also includes a smaller independently checked edit to `SidonPositions.lean`. It is deliberately not counted as an eighth mathematical direction. The source already uses the familiar sum-and-sum-of-squares argument to recover an unordered pair; the new edit changes the route through the polynomial identity, rather than introducing that mathematical fact.

After moving the relevant equalities to integers, use the identity

$$2(a-c)(a-d) = (a^2+b^2-c^2-d^2) + (a-b-c-d)(a+b-c-d).$$

If the first two moments agree, both terms on the right vanish. The resulting zero product yields $a = c$ or $a = d$, and the sum equation recovers the other coordinate. A deterministic `linear_combination` step avoids separately deriving the intermediate equality of products.

The particular identity is the proposed delta. Generic cast normalization with `zify`, polynomial normalization, and the original factorization argument are not claimed as new ideas. This narrow framing matters because earlier packets already discuss cast cleanup, coefficient fibres, and the existing Sidon interface.

The exact theorem declaration changes from 26 to 17 physical lines; the full candidate module changes from 168 to 161 lines after charging two explicit imports. Both the isolated theorem check and the complete modified `SidonPositions.lean` module completed successfully against the pinned dependencies. The logs and full-module receipt are supplied under `arithmetic/evidence`. This additional full-module check does not imply that the other production patches were built.